

AI Agent



AI Agent Workshop Preparation

Setup Environment

1. Clone project <https://github.com/je-labs.com/oleg-tsibulevskiy/AI-Agent-Workshop> if you haven't done so already
2. Environment Setup :
 - a. `python3 -m venv env` - Creates a virtual environment
 - b. `source env/bin/activate` - Activates virtual environment
 - c. Test, run `which python3` command, it should return the path with the `/env/bin/python3` in the end

3. Create a `requirements.txt` file and add required dependencies

```
1 fastapi==0.119.0          # The web framework used to build your API/Backend
2 uvicorn==0.32.1           # The lightning-fast server that runs your FastAPI
  application
3 httpx==0.28.1             # A modern client for making HTTP requests (needed for
  calling external APIs)
4 tqdm==4.67.1              # Adds those nice visual progress bars to your
  loops/downloads
5 langchain==1.2.2           # The "glue" framework that helps orchestrate LLMs,
  prompts, and chains
6 langchain-ollama==1.0.1    # The specific bridge to run local models (like Llama 3)
  via Ollama
7 langchain-chroma==1.1.0    # The integration for ChromaDB, your local Vector Database
8 pymupdf==1.26.7           # A high-performance library to read and extract text from
  PDF files
9 langchain-text-splitters==1.1.0 # Tools to chop long documents into smaller "chunks" for
  RAG
```

4. `pip3 install -r requirements.txt` - Install dependencies

If you have an error, try this command `source venv/bin/activate && pip3 install -r requirements.txt`

For the API creation, we will use the FastAPI framework.

Create a `main.py` file

```
1 from fastapi import FastAPI, HTTPException
2 from pydantic import BaseModel
3 from AIAgent import AIAgent
4
5 app = FastAPI()
6 agent = AIAgent()
7
8 class StartChatRequest(BaseModel):
9     message: str
10
11 class StartChatResponse(BaseModel):
12     response: str
13
14 import asyncio
15
16 @app.post("/chat", response_model=StartChatResponse)
17 async def start_chat(request: StartChatRequest):
18     response = await agent.llm_call(request.message)
19     if isinstance(response, str) and response.startswith("Error in LLM
  call:"):
20         raise HTTPException(status_code=400, detail=response)
21     return StartChatResponse(response=response)
```

`start_chat` is the API method responsible for accessing our Agent

AI Agent

Let's start with agent creation. Create the `AIAgent.py` file

```
1 from typing import Dict, Any
2 from langchain_ollama import ChatOllama
3 from langchain_core.messages import HumanMessage, ToolMessage,
  SystemMessage
4 from tools_provider import get_available_tools
5 import importlib
6
7
8 class AIAgent:
9     def __init__(self, model_name: str = "qwen2.5:7b"):
10         self.model_name = model_name
11         self.client = ChatOllama(model=model_name)
12         self.tools = get_available_tools()
13         self.client_with_tools = self.client.bind_tools(self.tools)
14
15     async def llm_call(self, prompt: str) -> str:
16         messages = [
17             SystemMessage(content="""You are a helpful data assistant.
18 When you receive data from tools:
19     - If asked for details, summarize the relevant information
20     - Respond in simple, natural language as if talking to a
21 person"""),
22             HumanMessage(content=prompt),
23         ]
24         try:
25             response = await self.client_with_tools.ainvoke(messages)
26
27             """
28             TOOL CALLS RESPONSE EXAMPLE:
29             {
30                 "tool_calls": [
31                     {
32                         "id": "call_001",
33                         "name": "get_weather",
34                         "args": {"location": "Paris"}
35                     }
36                 ]
37             }
38             """
39
40             if response.tool_calls:
41                 messages.append(response)
42
43                 for tool_call in response.tool_calls:
44                     function_name = tool_call["name"]
45                     arguments = tool_call["args"]
46                     print(f"🔍 Executing function: {function_name}
47 with arguments: {arguments}")
48                     tool_response =
49 self.execute_function(function_name, arguments)
50                     messages.append(ToolMessage(
51                         content=tool_response,
52                         tool_call_id=tool_call["id"]
53                     ))
54                 try:
55                     final_response = await
56 self.client_with_tools.ainvoke(messages)
57                     return final_response.content
58                 except Exception as e:
59                     return f"Error in final LLM call: {str(e)}"
```

```

57
58         return response.content
59
60     except Exception as e:
61         return f"Error in LLM call: {str(e)}"
62
63     def execute_function(self, function_name: str, arguments:
Dict[str, Any]) -> str:
64         try:
65             module = importlib.import_module(f"tools.{function_name}")
66             func = getattr(module, function_name, None)
67             if func is None:
68                 return f"Error: Function '{function_name}' not found
in module"
69             result = func(**arguments)
70             return str(result)
71         except ImportError as e:
72             return f"Error: Could not import module 'tools.
{function_name}': {str(e)}"
73         except Exception as e:
74             return f"Error: Function execution failed - {str(e)}"
75

```

Tools

Tools are methods or MCP services. The tool has access to external functions and data. For example, we have a service that provides information about Orders or some company information, and the creation of a tool that has access to this service will allow us to connect it to LLM. Having access to the tool LLM can answer any questions related to it and quickly analyze and provide the information.

OpenAI offers a way to define tools using a JSON Schema template

```

1  {
2      "type": "function",
3      "function": {
4          "name": "your_function_name",
5          "description": "A brief description of what your function does.",
6          "parameters": {
7              "type": "object",
8              "properties": {
9                  "parameter_name_1": {
10                     "type": "string",
11                     "description": "Description of parameter 1."
12                 },
13                 "parameter_name_2": {
14                     "type": "integer",
15                     "description": "Description of parameter 2."
16                 }
17             },
18             "required": ["parameter_name_1"]
19         }
20     }
21 }

```

Let's create our first tool. The tool will return static data, but it could be replaced with any API call or any other logic.

Create a new folder, `tools`.

Create a new file, `get_weather.py`

```
1 def get_weather(location: str, unit: str = "celsius") -> str:
2     """Mock weather function"""
3     return f"The weather in {location} is 22°{unit[0].upper()}, sunny"
```

Now we need to connect our tool to OpenAI

Create a new file in the main project directory, `tools_provider.py`

```
1 def get_available_tools():
2     return [
3         {
4             "type": "function",
5             "function": {
6                 "name": "get_weather",
7                 "description": """
8                 Get current weather for a location. This tool
9                 returns natural, conversational
10                weather information. The response includes
11                temperature, conditions, and friendly advice.
12                """
13                 "parameters": {
14                     "type": "object",
15                     "properties": {
16                         "location": {
17                             "type": "string",
18                             "description": "City name or location"
19                         },
20                         "unit": {
21                             "type": "string",
22                             "enum": ["celsius", "fahrenheit"],
23                             "description": "Temperature unit"
24                         }
25                     }
26                 },
27                 "required": ["location"]
28             }
29         }
30     ]
```

The `self.tools = get_available_tools()` code line is responsible for connecting tools to the OpenAI.

Now we can run the project and test our tool

To run the project call

```
1 uvicorn main:app --reload
```

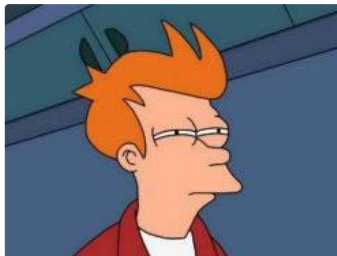
Now we can send a request to our server

```
1 | curl -X POST http://127.0.0.1:8000/chat -H "Content-Type: application/json" -d '{"message": "What is the weather in Winnipeg?"}'
```



Retrieval-Augmented Generation (RAG)

Retrieval-Augmented Generation (RAG) is the process of optimizing the output of an LLM, so it references an authoritative knowledge base outside of its training data sources before generating a response.



Let's move to [slides](#) 😊

First, we need to upload data to the Vector DB and break it into chunks. The Embedder transforms human language into mathematics — Vectors. For example, we have an Annual Report PDF, and we need to upload it to the Vector DB. Since it's a large document with multiple

pages, we break it into smaller chunks, so each section of the report will have its own chunk in the Vector DB.

For example, a chunk from the Annual Report PDF:

```
1 Revenue Growth: In fiscal year 2025, FastFeast achieved
2 total revenue of $142 million, representing a 23%
3 year-over-year increase. Our expansion into new markets
4 contributed significantly to this growth...
```

The Embedder will convert it to something like

```
1 [ 0.007834925, -0.015112009, 0.03297645, -0.000492864, -0.02103567,
   0.04562778, 0.01980753, -0.00392742, 0.005129642...]
```

Why do we need this? For example, we have an Annual Report with dozens of pages covering financial data, market analysis, and strategic plans — you can imagine how large this document is.

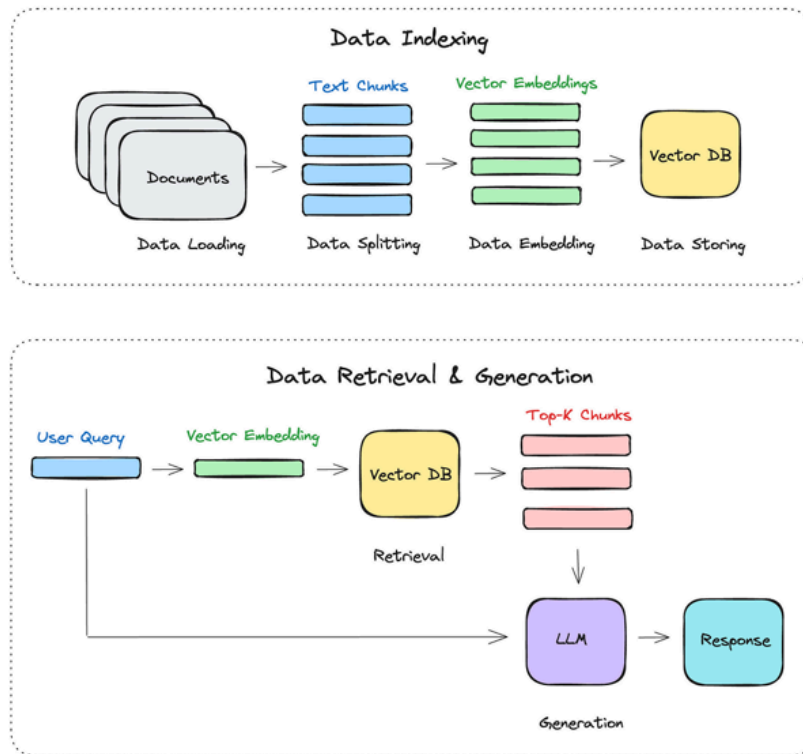
We don't want to pass all this data to the LLM to filter it, and we actually can't do this because we have a limit of input tokens. Oh, right, tokens! Tokens cost money, and we are paying for input and output, and RAG is the tool that will help us with this problem.

User sends a request, "What was the revenue in 2025?" The Vector DB will process the request and return the most relevant chunks from the annual report. For example, we defined in the query to return a maximum of 5 chunks.

Agree, this is much less information that will be sent to LLM for processing.

Result, fewer tokens/money spent, LLM returns a more accurate response

Basic RAG Pipeline



Find Restaurants Tool and Vector DB implementation

Create a `vector_store.py` file in the `rag` folder

```
1 from langchain_chroma import Chroma
2 from langchain_ollama import OllamaEmbeddings
3
4 db_location = "./chrome_db"
5 embeddings = OllamaEmbeddings(model="mxbai-embed-large")
6 db = Chroma(collection_name="michelin_restaurants",
7             persist_directory=db_location, embedding_function=embeddings)
8 top_k = 5
9 retriever = db.as_retriever(search_kwargs={"k": top_k})
```

Upload data to the DB

Create an `ingest.py` file in the `rag` folder

```
1 from pathlib import Path
2 from tqdm import tqdm
3 from vector_store import db
4 from langchain_core.documents import Document
5 import fitz
6 from langchain_text_splitters import RecursiveCharacterTextSplitter
7
8 def index_pdf_chunked(pdf_filename: str, doc_type: str):
9     pdf_path = Path(__file__).parent / pdf_filename
```



```

10
11     if not pdf_path.exists():
12         print(f"❌ File not found: {pdf_path}")
13         return
14
15     # Extract all text from PDF
16     doc = fitz.open(pdf_path)
17     full_text = ""
18     for page in doc:
19         full_text += page.get_text() + "\n\n"
20     doc.close()
21
22
23     # Chunk the text
24     splitter = RecursiveCharacterTextSplitter(
25         chunk_size=500,
26         chunk_overlap=50,
27         separators=["\n\n", "\n", ". ", " "]
28     )
29     chunks = splitter.split_text(full_text)
30
31     # Index chunks
32     for i, chunk in enumerate(tqdm(chunks, desc=f"📦 Indexing {pdf_filename}")):
33         metadata = {
34             "source": pdf_filename,
35             "chunk": i + 1,
36             "total_chunks": len(chunks),
37             "doc_type": doc_type
38         }
39         doc_id = f"{pdf_filename}_chunk_{i + 1}"
40         db.add_documents(
41             [Document(page_content=chunk, metadata=metadata)],
42             ids=[doc_id]
43         )
44
45     print(f"✅ Finished indexing {pdf_filename} ({len(chunks)} chunks)")
46
47 if __name__ == "__main__":
48     index_pdf_chunked("FastFeast Annual Report 2025.pdf",
49                       "annual_report_2025")

```

`page_content` → This is our chunk, and the search mechanism will look at this content to find a needed chunk

`metadata` → Is for returning useful data

Run `python3 rag/ingest.py` command

DB Query

Create a `query.py` file in the `rag` folder

```

1 from rag.vector_store import retriever
2
3 def query_annual_report_data(query: str):
4     results = retriever.invoke(query)
5     return results

```

Tool

Create an `annual_report.py` file in the tools folder

```
1 from rag.query import query_annual_report_data
2
3 def annual_report(query: str) -> str:
4     try:
5         results = query_annual_report_data(query)
6         if not results:
7             return "No relevant information found in the annual
8 report."
9
10        # Build response from relevant chunks
11        response_parts = [f"Found {len(results)} relevant section(s)
12in the annual report:\n"]
13
14        """
15        RESPONSE EXAMPLE:
16        {
17            "source": "FastFeast Annual Report 2025.pdf",
18            "chunk": 1,
19            "total_chunks": 10,
20            "doc_type": "annual_report"
21        }
22        """
23
24        for i, result in enumerate(results, 1):
25            content = result.page_content.strip()
26            metadata = result.metadata
27            chunk_num = metadata.get('chunk', '?')
28            total_chunks = metadata.get('total_chunks', '?')
29
30            response_parts.append(f"--- Section {i} (chunk
31{chunk_num}/{total_chunks}) ---")
32            response_parts.append(content)
33            response_parts.append("")
34
35        return "\n".join(response_parts)
36
37    except Exception as e:
38        return f"Error searching annual report: {str(e)}"
```

Update the `tool_provider` file

```
1 def get_available_tools():
2     return [
3         {
4             "type": "function",
5             "function": {
6                 "name": "get_weather",
7                 "description": """
8                 Get current weather for a location. This tool
9                 returns natural, conversational
10                weather information. The response includes
11                temperature, conditions, and friendly advice.
12                """,
13                 "parameters": {
14                     "type": "object",
15                     "properties": {
16                         "location": {
17                             "type": "string",
```

```

16         "description": "City name or location"
17     },
18     "unit": {
19         "type": "string",
20         "enum": ["celsius", "fahrenheit"],
21         "description": "Temperature unit"
22     }
23 },
24 "required": ["location"]
25 }
26 }
27 },
28 {
29     "type": "function",
30     "function": {
31         "name": "annual_report",
32         "description": ""
33         Use this tool to search the FastFeast Annual
34         Report 2025 for business information.
35         IMPORTANT: After receiving results, analyze
36         the content and respond in NATURAL LANGUAGE.
37         - Answer questions about revenue, growth,
38         performance, strategy, etc.
39         - Summarize findings from the report sections
40         - Provide specific data points when asked
41         "",
42         "parameters": {
43             "type": "object",
44             "properties": {
45                 "query": {
46                     "type": "string",
47                     "description": "Natural language query
48                     about the annual report. Can include questions about revenue, profits,
49                     growth, strategy, performance, quarters, etc."
50                 }
51             },
52             "required": ["query"]
53         }
54     }
55 }
56 ]

```

Call `curl -X POST http://127.0.0.1:8000/chat -H "Content-Type: application/json" -d '{"message": "What the revenue commission from delivery fees in 2025 compared with 2024?"}'`

The response should be similar to

```

{"response": "The revenue from delivery fees and commission increased
from $2,910 million in 2024 to $3,150 million in 2025. This shows a
growth of $240 million year-over-year."}

```